

I. Document récapitulatif

1. Présentation du projet

L'application est un jeu de Blackjack développé avec Symfony 7 et une API REST. Elle permet aux utilisateurs de :

- Créer un compte
- Se connecter via JWT
- Jouer des parties de Blackjack contre une IA
- Consulter leur profil

2. Architecture backend

Le backend suit l'architecture standard Symfony:

- Controllers : gestion des requêtes HTTP
- Services : logique métier (Blackjack, scores, règles)
- Entities : représentation des données (User, Game, Card...)
- Repositories : accès base de données via Doctrine
- Security (JWT) : gestion authentification et autorisations

Cette organisation facilite la mise en place des tests.

3. Fonctionnalités

L'audit du projet a permis d'identifier les fonctionnalités suivantes :

- Inscription utilisateur
- Authentification JWT
- Gestion du profil utilisateur
- Création de parties de Blackjack
- Déroulement d'une partie (hit / stand)
- Calcul des scores
- Détermination du gagnant
- Gestion des rôles (USER / ADMIN)

4. Analyse des risques

Certaines fonctionnalités présentent un niveau de risque plus élevé en raison de leur importance dans le fonctionnement de l'application.

Les fonctionnalités critiques sont :

- L'inscription des utilisateurs;
- L'authentification;
- La création d'une partie;
- La logique métier du Blackjack ;
- La gestion des autorisations selon les rôles.

Voici les risques identifiés concernant :

- Mauvaise validation des données envoyées par les clients ;
- Problèmes de sécurité sur les routes protégées ;
- Mauvaise gestion des droits administrateurs ;
- Erreurs dans le calcul des scores ou des règles du Blackjack ;
- Mauvaise gestion des erreurs HTTP.

5. Plan de test retenu

Au regard de cette analyse, le choix s'est porté sur un plan de test suivant.

- Tests unitaires afin de vérifier la logique métier contenue dans les services ;
- Tests fonctionnels Symfony afin de vérifier le comportement des contrôleurs et des routes ;
- Tests d'API afin de contrôler les réponses HTTP, les codes de retour, les validations et les autorisations.

6. Conclusion

L'architecture du projet est globalement claire et bien structurée, ce qui facilite l'analyse et les tests. Les fonctionnalités critiques ont été identifiées afin de concentrer les efforts de validation sur les éléments ayant le plus d'impact sur le fonctionnement de l'application. Cette analyse a servi de base à l'élaboration du plan de test et à la rédaction des différents scénarios de validation du backend.

II. Bugs trouvés

1. Absence de vérification `getUser()` → risque de null et erreur 500
2. Mauvaise gestion des erreurs via `\Error` au lieu d'`Exception`
3. Mauvaise validation des données JSON
4. Incohérence dans la logique métier du Turn sur le fichier `TurnController`
5. Gestion fragile des rôles admin dans `GameController`

III. Bugs corrigés

1. Sécuriser l'utilisateur connecté
2. Validation JSON
3. Remplacer `\Error` par `Exception`
4. Sécuriser logique Turn
5. Sécuriser rôle admin

IV. Tests écrits

1. `UserControllerTest`
2. `TurnControllerTest`
3. `GameControllerTest`
4. `BlackjackFlowTest`